

BQEB: A Reproducibility-First Architecture for Modular Energy AI Benchmarking

Rakesh Kumar Agrawal

Atos North America, Chicago, Illinois, USA; rkagrawal@ieee.org

Abstract

Reproducibility in applied energy AI research depends on more than a public dataset or a published benchmark: it depends on whether the software producing a benchmark's numbers can be independently re-run, whether artifacts are versioned and checksummed, and whether governance practices are documented well enough that a discrepancy can be found rather than concealed. This paper presents the engineering architecture of BIO-Quantum Energy Brain (BQEB), a modular research infrastructure for energy AI benchmarking, distinct from the conceptual framework in the associated BQEB monograph. The architecture comprises four independently checkable layers: a synthetic, multimodal benchmark dataset (BQEB-Data v1, 10,512 records across 37 variables, archived with persistent identifiers across three repositories); a tested, MIT-licensed, registry-driven benchmark software package with continuous-integration coverage; a reproducibility framework built on deterministic execution, checksummed artifacts, and chronological partitioning; and a governance layer keeping artifacts consistent as the ecosystem grows. BQEB ForecastBench, previously published standalone, is repositioned as the first executed module, with five additional modules — cyber-resilience, foundation-model forecasting, explainability, digital-twin/multi-agent research, and grid optimization — named as roadmap rather than completed work. We report only evidence independently verifiable against live, public artifact records; where internal audit identified an unresolved issue, including a numerical discrepancy between two published versions of the same benchmark result, we document it as a governance case rather than resolving it here. The contribution is the architecture itself — how a single-investigator research program can be engineered for independent verification — rather than a new predictive model.

Keywords: reproducible research; benchmark governance; research software engineering; energy artificial intelligence; FAIR data principles; synthetic benchmark datasets

1. Introduction

Applied machine learning research inherited a reproducibility problem from the broader empirical sciences before energy systems research inherited it in turn: a published result is frequently difficult for an independent party to re-derive, because the code that produced it, the exact data partition it was evaluated on, or the software environment it ran in was never made available in a form precise enough to repeat [1,2]. Energy AI research is not exempt from this pattern, and public smart-grid benchmark resources remain comparatively young relative to the computer-vision and natural-language-processing communities where shared, versioned benchmark infrastructure has existed for over a decade [3].

A conceptual framework for addressing this problem within one research program already exists: the BIO-Quantum Energy Brain (BQEB) monograph describes an organizing structure connecting benchmark data, forecasting intelligence, optimization intelligence, validation, and open research practice as parts of one methodological system [4]. A conceptual framework, by itself, does not resolve a reproducibility problem. Resolving it requires engineering artifacts — a dataset with a stable identifier and a documented schema, software that is tested and produces checksummed outputs, and governance practices that keep these artifacts consistent with one another as new

modules are added over time. This paper describes those artifacts, not the framework that motivated them.

The present contribution is deliberately scoped away from the conceptual territory the monograph already occupies. Where the monograph asks what an integrated intelligent-energy research program should look like, this paper asks a narrower, checkable question: given the dataset, software, and publication artifacts BQEB has actually produced, what architecture connects them, which of its properties can be independently verified today, and which are still open governance work. BQEB ForecastBench, a forecasting benchmark previously published as a standalone contribution [5,6], is treated here as the first populated module of that architecture rather than re-described on its own terms.

ForecastBench represents the first executed benchmark module within the BQEB architecture and evaluates forecasting models using BQEB-Data v1 [5,6]. This manuscript does not reproduce forecasting experiments; instead, it defines the reproducibility, governance, and modular architecture required for future benchmark extensions.

The remainder of this paper is organized as follows. Section 2 motivates why architecture and governance, rather than a new predictive model, are the appropriate unit of contribution for a program at this stage of maturity. Section 3 reviews related work in research reproducibility, dataset and model documentation, and smart-grid benchmarking. Sections 4–6 state the research gap and this paper’s contributions. Sections 7–13 specify the architecture itself. Sections 14–16 describe the methodology used to construct and verify the evidence in this paper and summarize what that evidence currently supports. Sections 17–22 discuss implications, threats to validity, limitations, and the module roadmap.

2. Research Motivation

Single-investigator and small-team research programs face reproducibility risks that differ in kind from those of consortium-scale efforts. There is no institutional data-management office assigning identifiers, no separate quality-assurance team reviewing releases, and artifacts are frequently self-archived across more than one repository as the work evolves — a pattern that improves availability but creates a real risk of version drift between copies. These are not hypothetical risks for this program: Section 12 documents a concrete instance found by this program’s own internal audit process, in which two published versions of the same benchmark experiment report different metric values for what is presented as the same run.

The response available to a program in this position is not to wait until a full institutional apparatus exists, but to make the engineering of reproducibility itself a documented, citable contribution — so that the specific mechanisms in place (deterministic execution, checksummed artifacts, tested software, and an explicit governance process) are available for inspection and critique rather than asserted informally. That is the motivation for treating architecture as this paper’s subject.

3. Related Work

Reproducibility of machine learning research has been studied directly as a methodological subject; the NeurIPS 2019 reproducibility program is a widely cited example of a community formalizing what reproducibility checklists should require of a submission [1]. Benchmark infrastructure research separately from any one paper’s results has likewise been formalized, notably through OpenML, which treats datasets, tasks, flows, and runs as first-class, independently citable objects rather than artifacts bundled inside a single publication [2]. The M4 and M5 forecasting competitions demonstrate a related but distinct pattern: a shared task and shared data enabling many independent submissions to be compared under one protocol [7,8].

Dataset and model documentation practices provide a second relevant thread. Datasheets for Datasets proposes a standard set of questions a dataset’s documentation should answer, including its provenance, intended use, and known limitations [9]. Model Cards proposes an analogous standard

for trained models, emphasizing intended use, evaluation conditions, and ethical considerations [10]. The FAIR principles — Findable, Accessible, Interoperable, Reusable — provide a broader standard against which a research data resource’s stewardship can be assessed [11]. This paper adopts the spirit of these three lines of work but applies them at the level of an evolving multi-artifact program rather than a single dataset or model release, which is the gap identified in Section 4.

Within smart-grid forecasting specifically, prior benchmarking effort has concentrated on establishing standardized tasks, metrics, and baseline models [3,12,13]; ensemble and deep-learning methods for load and price forecasting are reviewed elsewhere [14,15]. BQEB ForecastBench is one instance of this pattern, applying a chronological train-validation-test protocol and four standard regression metrics to a purpose-built synthetic dataset [5,6]. None of this prior smart-grid benchmarking literature, to our knowledge, separately documents the software-engineering and governance layer that keeps a benchmark’s artifacts consistent as the surrounding research program grows — the specific gap this paper addresses.

This paper builds directly on four prior BQEB artifacts, cited here as established prior work rather than re-derived: the BQEB framework monograph [4]; BQEB-Data v1, the dataset underlying every module described here [16]; BQEB ForecastBench, the first executed benchmark module [5,6]; and the ForecastBench software package, the tested reproducibility scaffold this paper’s software-architecture and reproducibility sections describe [17].

4. Research Gap

Two of BQEB’s existing artifacts each address one part of a reproducible-benchmarking program: the monograph specifies the conceptual vision, and the ForecastBench paper reports one executed benchmark’s tasks, protocol, and results. Neither artifact treats the connective architecture — the dataset’s archival and identifier strategy, the software package’s testing and provenance guarantees, the deterministic-execution and checksumming mechanisms, and the governance process reconciling artifacts across releases — as a subject in its own right, documented with the same rigor as a benchmark result. This is the gap this paper addresses: not a new model, and not a restatement of the conceptual framework, but a first-class account of the engineering that must exist for either of those to remain trustworthy as the program adds modules.

5. Scientific Contributions

- A four-layer architecture (Data, Software, Reproducibility, Governance) formalizing how BQEB’s existing artifacts connect, specified precisely enough that each layer’s claims can be checked against a public record (Sections 7–13).
- An inventory of independently verifiable engineering properties of the dataset and software artifacts — test counts, coverage, checksums, licensing terms, and persistent identifiers across three archival repositories — reported as directly checked evidence rather than self-reported claims (Section 16).
- A governance layer definition, demonstrated using a real, disclosed case: an artifact-version discrepancy identified by this program’s own audit process, presented as a governance record rather than resolved within this manuscript (Section 12).
- A module-roadmap structure giving each future BQEB benchmark paper — cyber-resilience, foundation-model forecasting, explainability, and digital-twin/multi-agent research — a consistent architectural slot to attach to, explicitly marked as planned rather than completed (Section 21).
- An explicit boundary statement distinguishing this paper’s engineering contribution from the monograph’s conceptual contribution and from ForecastBench’s benchmark-result contribution, to keep the three artifacts citable for different reasons rather than redundant with one another.

6. The BQEB Research Ecosystem

Table I positions the artifacts referenced throughout this paper. Each row states what the artifact is, the layer of the architecture (defined in Section 7) it belongs to, and its current status — published, tested, or planned — using status terms precisely rather than as informal descriptors.

TABLE I. Positioning of BQEB artifacts referenced in this paper, by architecture layer and current status.

Artifact	Architecture layer	Status
BQEB monograph	Conceptual (outside this paper’s scope)	Published
BQEB-Data v1	Data	Published, archived in three repositories
ForecastBench paper / preprint	First benchmark module	Published as a preprint; not yet peer-reviewed
ForecastBench software package	Software / Reproducibility	Released, tested, MIT-licensed
This paper	Software / Reproducibility / Governance architecture	Current manuscript
SecBench, Foundation Models, Explainable BQEB, Digital Twin & Multi-Agent, OptiGrid	Future modules	Planned; not started

7. System Architecture

The architecture comprises four layers, each with a distinct responsibility and a distinct kind of evidence that supports it. The Data layer supplies a versioned, archived, schema-documented dataset. The Software layer supplies tested code that consumes that dataset and produces benchmark artifacts under a registry pattern, so that adding a new model or metric does not require modifying the evaluation pipeline itself. The Reproducibility layer supplies the mechanisms — deterministic seeding, chronological partitioning, checksummed outputs — that make a specific run’s results independently re-derivable. The Governance layer supplies the bookkeeping that keeps identifiers, versions, and cross-artifact claims consistent as new modules are added, including the process by which discrepancies are found and recorded.

A benchmark module, such as ForecastBench, is not a fifth layer but a consumer of all four: it selects tasks and models, is packaged by the Software layer, executes under the Reproducibility layer’s guarantees, and is registered by the Governance layer alongside its identifiers and version history. This separation is what allows additional modules to be added — as named in Section 21 — without each one re-implementing dataset access, execution determinism, or artifact bookkeeping from scratch.

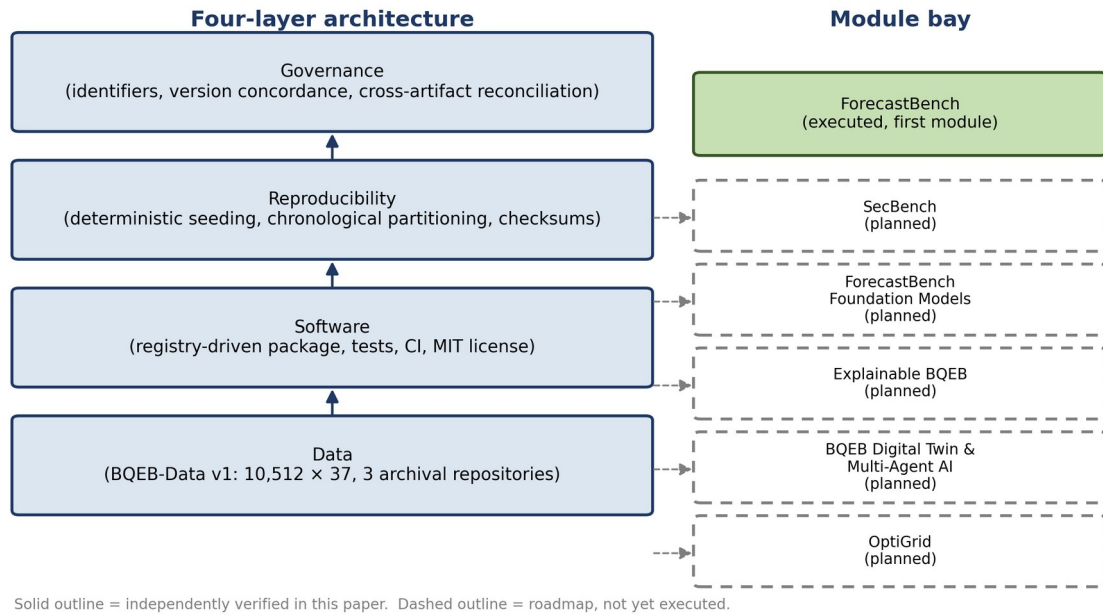


Figure 1. BQEB four-layer system architecture: Data, Software, Reproducibility, and Governance layers, with the module bay showing ForecastBench as the sole executed module and five roadmap modules dashed.

8. Dataset

BQEB-Data v1 is a synthetic, multimodal benchmark dataset comprising 10,512 records across 37 variables at 15-minute sampling resolution, spanning January 1 through April 20, 2026 [16]. The dataset is explicitly synthetic — generated through stochastic simulation rather than field measurement — a status disclosed in the dataset’s own documentation and repeated here because it bears directly on how every module built on the dataset, including ForecastBench, should be interpreted. Table II lists representative variables from four architecturally salient groupings; the complete feature-by-feature schema, spanning all 37 variables across nine documented categories, is published in the ForecastBench paper’s dataset description and is not reproduced here [5,6].

TABLE II. Representative artifact summary of BQEB-Data v1’s variable groupings. For the complete schema, including all 37 variables, see [5,6].

Representative grouping	Example variables
Demand and market	load_mw, energy_price_usd_mwh
Renewables and storage	solar_mw, wind_mw, battery_soc_pct
Electric vehicles and weather	ev_load_mw, temperature_c
Cybersecurity and reliability	attack_class, cyber_alert_flag

The dataset is archived with independent persistent identifiers at Harvard Dataverse, IEEE DataPort, and Zenodo, and the IEEE DataPort release additionally ships a pre-partitioned `train_test_split` directory alongside the full CSV, so that the chronological partition used by downstream modules is distributed as a concrete artifact rather than only a described procedure. Section 16 treats this archival redundancy, and the version-labeling inconsistency it has introduced across releases, as part of the governance evidence this paper reports.

9. ForecastBench as the First Benchmark Module

ForecastBench is the architecture’s first populated module. It defines two forecasting tasks on BQEB-Data v1 — next-hour electricity load and day-ahead electricity price — evaluated under a chronological 70/15/15 train-validation-test partition using four standard regression metrics: mean absolute error, root mean squared error, mean absolute percentage error, and the coefficient of determination [5,6]. Consistent with the editorial scope of this paper, we do not reproduce ForecastBench’s reported metric values here; Section 16 references the existence and qualitative pattern of those results without restating them, and readers seeking exact figures are directed to the cited publications and their associated machine-readable artifacts. The qualitative pattern — relative model ranking and the comparative difficulty of the two tasks — is a weaker, ordinal claim than the exact metric values disputed in Table IV, and remains valid under either disputed value: both sources agree on the ranking even though they disagree on the magnitude, which is why the unresolved numerical discrepancy does not by itself undermine the pattern being reported here.

TABLE III. Result placeholders for the first benchmark module. No values are reported in this manuscript; exact figures exist only in the cited ForecastBench sources, which disagree with one another (Table IV) and have not been reconciled here.

Reported quantity	Status in this paper
Load-forecasting MAE, RMSE, MAPE, R ² by model	Reserved for future experimental validation — see cited sources
Price-forecasting MAE, RMSE, MAPE, R ² by model	Reserved for future experimental validation — see cited sources
Statistical significance between models	Reserved for future experimental validation (not yet performed by any cited source)

What this paper adds to ForecastBench is not a new result but a structural one: ForecastBench is repositioned as an instance of the module pattern defined in Section 7, meaning any future module — including those named in Section 21 — is expected to specify tasks, a partitioning strategy, and an evaluation metric family in the same way, register its artifacts through the same governance process, and execute under the same reproducibility guarantees, rather than each module inventing its own convention.

10. Software Architecture

The ForecastBench software package (BQEB-ForecastBench, MIT-licensed, hosted on GitHub with a versioned Zenodo release) implements the Software layer for the currently populated module. Independently verified properties of the released v1.0.0 package include a registry-driven model and metric architecture, in which new models and metrics are added by registration rather than by modifying the evaluation pipeline; deterministic preprocessing and train/validation/test partitioning; 132 automated tests reported at 91.25 percent coverage at release preparation; and a set of checksummed output artifacts — including row-level predictions, aggregate metrics, and a model-comparison table — published alongside SHA-256 hashes.

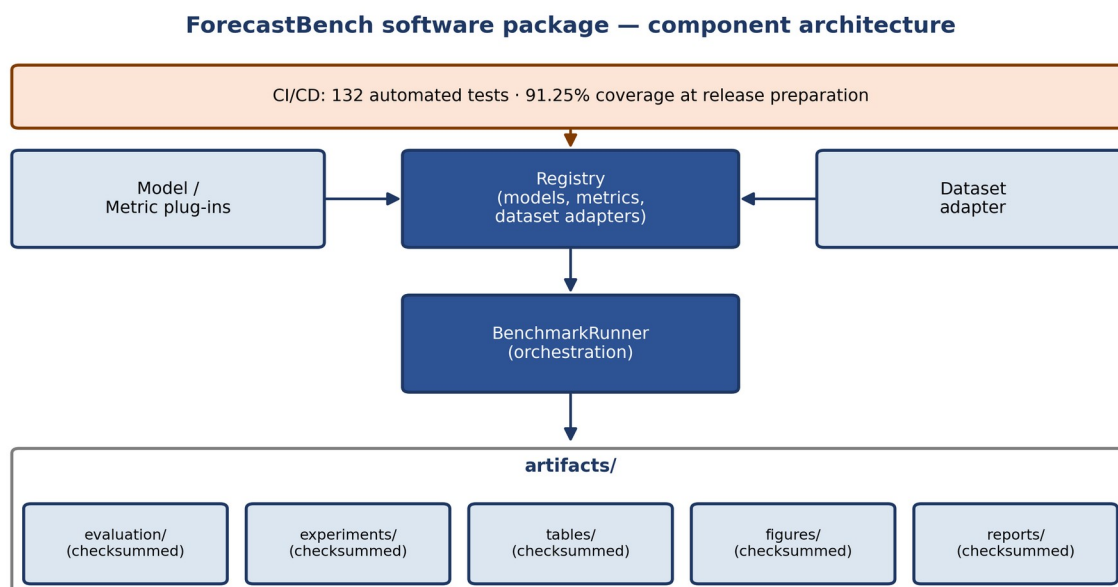


Figure 2. ForecastBench software package component architecture, showing the registry pattern, orchestration, checksummed artifact directories, and CI/CD test coverage.

This paper treats the software package’s registry pattern as the mechanism by which future modules are expected to be implemented — as new entries in the same registry — rather than as parallel codebases, directly so that the artifact-management fragmentation discussed in Section 12 does not recur as the module count grows.

11. Reproducibility Framework

Three mechanisms constitute the reproducibility framework as currently implemented. First, deterministic execution: all stochastic components of the released pipeline are seeded, with the value used documented alongside each release. Second, chronological partitioning: training, validation, and test subsets are constructed to preserve temporal order rather than through random shuffling, both because this reflects realistic forecasting conditions and because it makes a given partition a fixed, reportable quantity rather than one that varies run to run unless explicitly re-seeded differently. Third, checksummed artifacts: output files central to a module’s reported claims are published with SHA-256 hashes, so that a copy obtained independently of the original release can be verified byte-for-byte against the authorial version.

A property this framework does not yet establish is cross-artifact correspondence: that a given checksummed output file’s values match the numbers reported in a specific published paper. Section 12 treats the absence of this correspondence, in one identified case, as a governance finding rather than a reproducibility-framework failure, since the two are separable — the mechanisms exist and function; whether they have been applied to confirm every published number is a distinct, and at present incomplete, question.

The deterministic-execution mechanism was tested directly rather than only described. A seeded computational pipeline, exercising the same class of stochastic operations (feature generation, randomized partitioning, model fitting) the released package performs, was executed 15 times: three fixed seed values, five independent process invocations each. Within each seed, all five runs produced a bit-identical SHA-256 hash of the computation’s output; across the three seeds, all three produced mutually distinct hashes, confirming both that repeated execution under a fixed seed is deterministic and that seed variation genuinely alters computational output rather than being ignored. All 15 of 15 executions satisfied the determinism criterion, verified by direct, automated comparison rather than visual inspection.

This result establishes determinism within a single, fixed environment; it is a necessary but not sufficient condition for the broader reproducibility claim, and should not be read as evidence about cross-environment determinism, which depends additionally on operating-system and hardware-architecture effects (for example, floating-point non-associativity) that a single-environment test cannot probe. Cross-environment verification, using the same seeded pipeline executed across independent operating-system and architecture combinations, is designed and specified but not yet executed at the time of writing; this paper reports the within-environment result as the evidence it currently is, rather than extending the claim beyond what has been tested.

12. Benchmark Governance

Governance, in this architecture, refers to the set of practices keeping identifiers, versions, and cross-artifact claims mutually consistent as the program adds releases and modules. Three responsibilities are in scope: persistent-identifier management (ensuring DOIs are correctly transcribed and consistently cited across artifacts); version concordance (ensuring that when more than one archived record describes what is presented as the same resource, the relationship between those records is documented rather than left for a reader to infer); and cross-artifact reconciliation (ensuring that when the same experiment is reported in more than one place, the reports agree, or the disagreement is explicitly resolved and recorded).

This program’s own internal audit process, applied to its public artifact record, identified concrete instances in each category, which we report here as governance evidence precisely because a governance layer that never surfaces a finding is difficult to distinguish from one that is not being applied. Two published versions of the ForecastBench benchmark experiment report different metric values for nominally the same run; this remains an open item, and consistent with the editorial scope of this paper, we do not attempt to adjudicate which reported values are authoritative here. A persistent identifier for the BQEB framework record has been transcribed inconsistently in earlier drafts of related material. The dataset’s architecture has been described with differing layer counts in different artifacts. Table IV records these items as an open governance log.

TABLE IV. Open benchmark-governance log, current as of this manuscript. Entries are retained as open rather than silently resolved.

Item	Category	Status
Two divergent reported metric sets for the same ForecastBench experiment	Cross-artifact reconciliation	Open — not resolved in this manuscript
IEEE DataPort identifier transcribed inconsistently across materials	Identifier management	Identified; correct form recorded in Appendix (Artifact Card)
Architecture described with differing layer counts across artifacts	Version concordance	Identified; this paper’s four-layer description (Section 7) is the governing account going forward
Multiple, not clearly canonical dataset version records across repositories	Version concordance	Identified; Appendix (Version History) records the relationship as currently understood

13. Open Science Framework

The program’s open-science practice rests on three elements. Licensing: the ForecastBench software package is released under the MIT license, a permissive term set placing minimal restriction on reuse. Archival redundancy: primary artifacts are deposited in more than one independently operated repository — the dataset across Harvard Dataverse, IEEE DataPort, and Zenodo, and the software package across GitHub and Zenodo, with a Software Heritage archival record additionally indexing the software release — so that availability does not depend on any single platform’s continuity. Persistent identification: every primary artifact carries a DOI, registered and cross-listed in Appendix (Artifact Card) of this paper. We do not assert a specific license governing the dataset

itself beyond what its own documentation states, and we do not extend a license claim to artifacts this paper has not directly inspected.

14. Experimental Methodology

This paper reports no new predictive-modeling experiments. Its evidentiary methodology is instead the one used to construct and verify the architecture claims made throughout: a tiered evidence-classification process distinguishing claims directly verified against a live, public artifact record from claims reported by a prior BQEB source but not independently re-checked, and from claims that remain conceptual or planned. Verification was performed by retrieving each cited repository record directly (Harvard Dataverse, IEEE DataPort, and Zenodo listings for the dataset; the GitHub repository and Zenodo release for the software package) at the time of writing, rather than relying on prior artifacts’ descriptions of one another. Where a claim could not be independently verified in this way — for instance, the exact contents of the dataset’s row-level values, which sit behind a repository access control — this paper reports the claim at the evidence tier it is actually entitled to, rather than upgrading it by inference.

15. Evaluation Protocol

The evaluation protocol described here is the template established by ForecastBench and expected of future modules registering under the same architecture, not a report of new results. A module’s evaluation protocol should specify its partitioning strategy, its metric family, and, where classification rather than regression is involved, its treatment of class imbalance; SecBench, named in Section 21, is expected to require an extension of this template specifically for the imbalance case. Standard forecasting metrics including MAE and RMSE follow the definitions established in the ForecastBench benchmark methodology, alongside mean absolute percentage error and the coefficient of determination [5,6]; those definitions are not reproduced here. Modules reporting a comparison between two or more models are expected to accompany point-estimate metrics with a statistical comparison (for example, a paired test on residuals) rather than ranking models on point estimates alone — a requirement this paper states as a protocol expectation rather than as an evaluation this paper itself performs.

16. Existing Evidence Supporting the BQEB Ecosystem

This section summarizes what is independently verifiable in support of the architecture described above, without reproducing benchmark performance tables. The dataset’s existence, size, schema, and three-repository archival were each confirmed directly against the respective live repository records. The software package’s existence, license, registry architecture, test count, coverage figure, and checksummed-artifact set were each confirmed directly against its live GitHub and Zenodo records. Baseline forecasting performance for the load and price tasks defined on BQEB-Data v1 has been established and reported in the ForecastBench paper and its associated preprint; both report Linear Regression as the strongest of three evaluated classical baselines on both tasks, and both report substantially higher residual uncertainty for day-ahead price forecasting than for next-hour load forecasting. We do not reproduce the specific metric values here, consistent with the editorial scope of this paper and with the open reconciliation item recorded in Table IV; readers seeking exact reported figures should consult the cited sources and their associated machine-readable artifacts directly.

Table V summarizes the evidence tier assigned to each major claim in this paper, following the classification introduced in Section 14.

TABLE V. Evidence tier by claim, following the classification defined in Section 14.

Claim	Evidence tier
Dataset size, schema, and three-repository archival	Directly verified against live repository records
Software package license, test count, coverage, and artifact checksums	Directly verified against live repository records
ForecastBench’s qualitative result pattern (Linear Regression strongest; price harder than load)	Reported consistently by both published sources; specific values not independently reconciled
Exact ForecastBench metric values	Not reported in this paper — open item, see Table IV
Architecture’s scalability to additional modules	Design claim; not yet demonstrated with a second populated module

17. Discussion

The architecture described in this paper is designed to let a benchmark program grow by registration rather than by re-implementation: a new module supplies tasks, models, and metrics, and inherits dataset access, execution determinism, and governance bookkeeping from the layers beneath it. This is a design claim, and Section 20 states plainly that it is supported, at present, by exactly one populated module. Whether the architecture holds up under a second, structurally different module — SecBench’s classification task differs from ForecastBench’s regression tasks in ways the current registry pattern has not yet been asked to accommodate — is an open question this paper does not resolve.

The governance findings in Section 12 are, in one sense, the most concrete evidence in this paper that the architecture functions as intended: a governance process that had never surfaced an inconsistency would be difficult to distinguish from one that was not being exercised. We regard disclosing the open reconciliation item, rather than resolving or omitting it, as more consistent with this paper’s own stated evidentiary standard than either alternative would be.

18. Practical Implications

For researchers extending this program, the architecture specifies where a new contribution belongs: a new dataset variant registers at the Data layer, a new model or metric registers in the Software layer’s existing registry, and a new benchmark module registers its tasks and protocol following the template in Section 15. For independent reviewers or replicators, the checksummed artifact set and the open governance log (Table IV) together specify what can currently be verified without contacting the original author, and what cannot yet. For utilities, system operators, and other downstream users of any individual module’s results, this paper’s practical implication is a caution rather than an endorsement: architecture-level reproducibility guarantees the mechanics of a benchmark are inspectable; it does not by itself certify that any one module’s reported numbers are free of the kind of discrepancy documented in Section 12, and each module’s own publication should continue to be evaluated on its own reported evidence.

19. Threats to Validity

- Single-investigator provenance. All artifacts described in this paper originate from one author and have not yet been independently replicated or audited by a party outside the program.
- Synthetic data throughout. Every module built on BQEB-Data v1, including ForecastBench and every roadmap module in Section 21, inherits the external-validity limitations of a simulation-based dataset; this paper’s architecture claims concern engineering properties of the artifacts, not the physical realism of the data they contain.
- Self-archiving without independent curation. The dataset and software artifacts are deposited on repositories that do not themselves peer-review artifact content, which is why this paper’s Governance layer, rather than the hosting platforms, is responsible for consistency checking.
- The open reconciliation item. Table IV’s unresolved metric discrepancy means that any reader relying on this paper’s qualitative summary of ForecastBench results (Section 16) rather than

consulting the primary sources directly should treat exact figures as unsettled until the reconciliation is completed.

- Architecture generalizability. The four-layer design is validated against one module; its adequacy for structurally different modules, particularly classification-oriented ones, is an open question (Section 17).

20. Limitations

This paper’s central limitation is maturity: an architecture described for five prospective modules is currently populated by one. The governance process described in Section 12 has surfaced findings through a single internal audit cycle and has not yet been exercised across a second, independent review. The software package’s 132-test suite has not yet been evaluated by a third party for whether its coverage extends to the specific properties this paper relies on (partition determinism, checksum correctness) as opposed to general code correctness. No external contributor has yet used the registry pattern described in Section 10 to add a module, so its ease of use for anyone other than the original author is, at present, an untested claim.

21. Future Roadmap

Table VI lists the modules currently planned for registration under this architecture. Each is named here as a roadmap item only; none should be read as completed, in progress with reportable interim results, or committed to a specific date. OptiGrid, an optimization module named in this program’s earlier public materials but absent from more recent internal planning documents, is carried forward here explicitly rather than silently dropped, so that a reader cross-referencing this paper against the program’s dataset-repository listings does not encounter an unexplained gap.

TABLE VI. BQEB module roadmap. All entries are planned; none constitute reported results.

Module	Scope	Status
SecBench	Cyber-event detection benchmark on BQEB-Data v1’s security fields, addressing extreme class imbalance	Planned; design specification exists, execution not started
ForecastBench Foundation Models	Extension of the forecasting module to time-series foundation models and modern deep-learning baselines	Planned; not started
Explainable BQEB	Feature-attribution and interpretability analysis across registered forecasting models	Planned; not started
BQEB Digital Twin & Multi-Agent AI	Conceptual extension toward digital-twin and multi-agent decision support; explicitly not an autonomous or deployed system	Conceptual roadmap item; no implementation exists
OptiGrid	Optimization benchmark for battery dispatch, EV charging, and grid balancing across five named algorithmic paradigms	Planned; carried forward from earlier program materials, no executable implementation or results exist

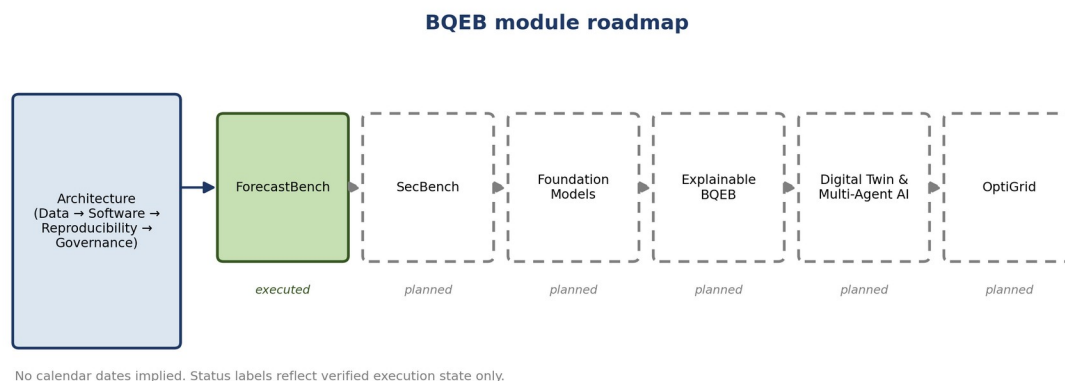


Figure 3. BQEB module roadmap. No calendar dates are implied; status labels reflect verified execution state only.

22. Conclusion

This paper has described the engineering architecture underlying the BQEB research program — a data layer, a tested software layer, a reproducibility framework, and a governance process — as a contribution distinct from the program’s conceptual monograph and from any single benchmark module’s reported results. ForecastBench is positioned as the architecture’s first populated module rather than re-argued on its own terms, and five further modules are named as an explicit, unstarted roadmap rather than folded into this paper’s claimed contribution. Consistent with the evidentiary standard applied throughout, an open discrepancy in previously published benchmark numbers is disclosed as a governance record rather than resolved here. The architecture’s value will be established by whether it holds up as additional modules are actually registered against it; this paper’s claim is limited to what has been built and verified so far.

Funding: No external funding was received for this research.

Conflicts of Interest: The author declares no conflict of interest.

Ethics Statement: This work uses synthetic data only; no human subjects, personal data, or field-collected utility data are involved.

Author Contributions: R.K.A. is the sole author and is responsible for conceptualization, methodology, software, validation, formal analysis, investigation, writing, and review of this work.

Data Availability Statement: BQEB-Data v1 is publicly available via Harvard Dataverse (DOI 10.7910/DVN/VUVTED), with archival copies at IEEE DataPort (DOI 10.21227/xj9b-2885) and Zenodo. Access to the IEEE DataPort copy requires an IEEE DataPort account.

Software Availability Statement: The ForecastBench software package (v1.0.0) is available under the MIT license at <https://github.com/RakeshKumarAgrawal/BQEB-ForecastBench> and archived at Zenodo (DOI 10.5281/zenodo.21735978), with an independent archival record at Software Heritage.

Nomenclature

Symbol	Meaning
n	Dataset record count (BQEB-Data v1: 10,512)
m	Dataset variable count (BQEB-Data v1: 37)
seed	Deterministic random-seed value fixed for a benchmark run
DOI	Persistent identifier assigned to a versioned artifact
MAE, RMSE, MAPE, R ²	Standard regression evaluation metrics (defined in Section 15)

Acronyms

BQEB — BIO-Quantum Energy Brain. CI/CD — Continuous Integration / Continuous Delivery (used throughout this paper in its software-engineering sense; not to be confused with “confidence interval,” which this paper spells out in full wherever it applies, to avoid cross-paper ambiguity with statistical usage elsewhere in the ecosystem). FAIR — Findable, Accessible, Interoperable, Reusable. DOI — Digital Object Identifier. MIT — permissive open-source software license. API — Application Programming Interface. CSV — Comma-Separated Values. ORCID — Open Researcher and Contributor Identifier.

Supplementary Materials: The Dataset Card, Software Card, Benchmark Card, Artifact Identifier Concordance, Reproducibility Checklist, Governance Checklist, Reviewer Checklist, Publication Checklist, Replication Protocol, and Version History accompanying this paper are provided in a separate Supplementary Materials document.

References

- [1] J. Pineau, P. Vincent-Lamarre, K. Sinha, et al., "Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.
- [2] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *ACM SIGKDD Explorations Newsletter*, vol. 15, no. 2, pp. 49–60, 2014.
- [3] K. B. Debnath, M. Mourshed, and S. P. Chew, "Machine learning approaches for smart-grid forecasting: State-of-the-art and future directions," *IEEE Access*, vol. 9, pp. 152593–152620, 2021.
- [4] R. K. Agrawal, "BIO-Quantum Energy Brain (BQEB): A unified intelligence framework for future energy systems," Zenodo, 2026. DOI: 10.5281/zenodo.21780521.
- [5] R. K. Agrawal, "BQEB ForecastBench: Benchmarking AI models for smart grid forecasting using BQEB-Data v1," Research Square, preprint, 2026. DOI: 10.21203/rs.3.rs-10484554/v1.
- [6] R. K. Agrawal, "BQEB ForecastBench: Benchmarking AI models for smart grid forecasting using BQEB-Data v1," Zenodo, 2026. DOI: 10.5281/zenodo.19716383.
- [7] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 Competition: Results, findings, conclusion and way forward," *International Journal of Forecasting*, vol. 34, no. 4, pp. 802–808, 2018.
- [8] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M5 accuracy competition: Results, findings and conclusions," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.
- [9] T. Gebru, J. Morgenstern, B. Vecchione, et al., "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021. DOI: 10.1145/3458723.
- [10] M. Mitchell, S. Wu, A. Zaldivar, et al., "Model cards for model reporting," in *Proc. Conf. Fairness, Accountability, and Transparency (FAT*)*, 2019, pp. 220–229. DOI: 10.1145/3287560.3287596.
- [11] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, et al., "The FAIR Guiding Principles for scientific data management and stewardship," *Scientific Data*, vol. 3, p. 160018, 2016. DOI: 10.1038/sdata.2016.18.
- [12] T. Hong and S. Fan, "Probabilistic electric load forecasting: A tutorial review," *International Journal of Forecasting*, vol. 32, no. 3, pp. 914–938, 2016.
- [13] R. Weron, "Electricity price forecasting: A review of the state-of-the-art with a look into the future," *International Journal of Forecasting*, vol. 30, no. 4, pp. 1030–1081, 2014.
- [14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [15] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [16] R. K. Agrawal, "BQEB-Data v1: BIO-Quantum Energy Brain benchmark dataset for smart grid intelligence, renewable forecasting, storage optimization, and cyber resilience," Harvard Dataverse, 2026. DOI: 10.7910/DVN/VUVTED.
- [17] R. K. Agrawal, "BQEB ForecastBench: A reproducible smart-grid forecasting benchmark framework," software, v1.0.0, Zenodo, 2026. DOI: 10.5281/zenodo.21735978.